Studying grammatical change with the Penn Parsed Corpora of Historical English
Session 3: Practical: More advanced CorpusSearch queries

George Walkden
george.walkden@uni-konstanz.de

1. CorpusSearch: a warm-up exercise

Open up CorpusSearch and try to find the following in the PPCEME: (Use node $ROOT.)

- The word consent used as a noun (N*)? (Hint: part-of-speech tags immediately dominate the word they tag.)
    - Query: _____
    - Number of hits: _____
- A quantifier (Q) immediately followed by an adjective (ADJ*)?
    - Query: _____
    - Number of hits: _____
- A noun phrase (NP*) that contains a clause (CP*)?
    - Query: _____
    - Number of hits: _____

2. More query functions

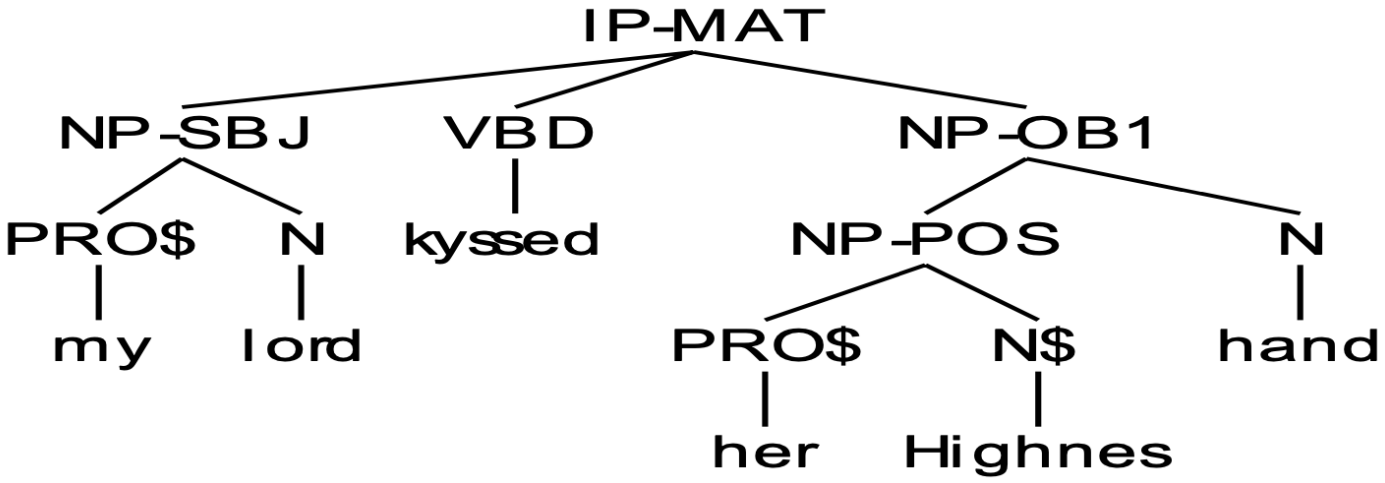
Yesterday we introduced the following functions:

- Exists
- hasSister
- Precedes
- iPrecedes
- iDominates (or iDoms)

Here are a few more handy functions:

- X CCommands Y
    - X c-commands Y.
    - For instance, in the below tree, NP-SBJ c-commands VBD, NP-OB1 and everything dominated by either, but not IP-MAT or anything that it dominates itself.
- X iDomsNumber Y Z
    - X immediately dominates Z as the Y^{th} child.
    - For instance, `(IP-MAT iDomsNumber 1 CONJ)` finds matrix clauses that start with a conjunction.

- X iDomsLast Y
  - X immediately dominates Y, and Y is the last child of X.
  - For instance, in the below tree, NP-OB1 is the last child of IP-MAT. N is the last child of NP-OB1. (For you to work out: what is the last child of NP-POS?)



3. Definition files

Earlier English is often thought to be a type of verb-second (V2) language. To find all clauses in which the verb comes second, you can use the iDomsNumber function.

```
IP-MAT* iDomsNumber 2 …??????
```

What we want is for a matrix clause (IP-MAT) to immediately dominate as its second child (iDomsNumber 2) the finite verb. But how do we search for the finite verb?

Some finite verb parts-of-speech in the Penn Historical Corpora of English:

- VBP (a present tense lexical verb)
- VBD (a past tense lexical verb)
- BEP (a present tense form of the verb ‘to be’)
- BED (a past tense form of the verb ‘to be’
- HVP (a present tense form of the verb ‘to have’)
- HVD (a past tense form of the verb ‘to have’)
- DOP (a present tense form of the verb ‘to do’)
- DOD (a past tense form of the verb ‘to do’)
- MD (a modal)
- and potentially more!

One possibility is to use the logical operator | (‘or’) and just string them all together:

```
IP-MAT* iDomsNumber 2 VBP|VBD|BEP|BED|HVP|HVD|DOP|DOD|MD
```

But it's not great to have to type this in every time you write a query: it's time-consuming and error-prone.

Instead, you can use a **definition file**. Open up Notepad or TextEdit. Copy and paste the below, and save it as verb.def in your CS folder:

```
finite_verb: VBP|VBD|BEP|BED|HVP|HVD|DOP|DOD|MD
```

Here, `finite_verb` has been **defined** as all of the above parts of speech. When you write a query, if you want to use finite verbs as a search term, you can simply include an extra line at the beginning of your query file:

```
define: verb.def
```

Then you can write the query using the definition you've already created. In this case:

```
IP-MAT* iDomsNumber 2 finite_verb
```

Definition files will save you time in the long run if you want to do lots of similar queries, and also guard against errors.

4. Other CorpusSearch tips and tricks

- If you want to search for two things at once, you can do that using AND. For instance:
```
query: (IP* iDoms NP-SBJ*)
AND (NP-SBJ* iDoms PRO*)
AND (finite_verb iPrecedes NP-SBJ*)
```

- CorpusSearch can also search its own output. In this case, you need to specify that the source file of the new query is the .out file produced by the old query. For instance, you might want to search first for all V2 clauses, and then for all V2 clauses that contain a direct object.
- **Complement** files are sometimes useful. To use this feature, add the following line to the beginning of your query file: `print_complement: t`
- This will cause CorpusSearch to produce two files.
    - One is the usual .out file which contains the structures that match your query.
    - One is a .cmp file which contains all the structures that do **NOT** match your query.
- This is useful if you want to search, for instance, for non-pronominal NPs, which are hard to search for. You simply need to set up your query to search for pronominal NPs, which are easier to search for. All other (non-pronominal) NPs will then appear in the complement file.

- **Same instance**: When two search terms match, they are forced to apply to the same node. Thus two uses of NP-OB* will require that, if for instance, NP-OB2 is found as an instance of the first NP-OB*, then the next use of NP-OB* will also apply to the **same** NP-OB2 (not, for instance, an NP-OB1 which may also be in the vicinity).
- In order to force non-same instance, use index numbers. [1]NP* and [2]NP* cannot apply to the same NP* node. For example, the query: `(IP* iDoms [1]NP*) AND (IP* iDoms [2]NP*)`
- This will find all clauses that immediately dominate two *different* noun phrases. Without the index numbers, it would also find all clauses that immediately dominate only a single noun phrase.

5. More exercises

Try to find the following in the PPCEME. Use node IP*, as before.

- All instances of the prepositions *to* and *from*?
  (Hint: part-of-speech tags immediately dominate the word they tag.)

  - Query: _____
  - Number of hits: _____

- All clauses where the subject is at the end?

  - Query: _____
  - Number of hits: _____

- All clauses without an indirect object?

  - Query: _____
  - Number of hits: _____

- All clauses that immediately contain three different prepositional phrases?

  - Query: _____
  - Number of hits: _____

Answers to exercises

- All instances of the prepositions *to* and *from*? `(P iDoms to|from)`
  - NB: You can't use AND for this. Can you figure out why?
- All clauses (IP*) where the subject is in final position? `(IP* iDomsLast NP-SBJ*)`
- All clauses without an indirect object? `(IP* iDoms !NP-OB2*)`
  - NB: this can also be done with a complement file
- All clauses that immediately contain three different prepositional phrases?

```
(IP* iDoms [1]PP) AND (IP* iDoms [2]PP) AND (IP* iDoms [3]PP
```